

Serverless Application Model de AWS

▼ Serverless Application Model de AWS

▼ Introsucción

Ventajas frente a CloudFormation puro

SAM como extensión de CloudFormation (Transform)

▼ AWS SAM CLI

Instalación y configuración inicial

Inicializar un proyecto con `sam init`

Estructura de un proyecto SAM después de `sam init`

Construir la aplicación localmente con `sam build`

Pruebas locales con SAM CLI con `sam local`

Desplegar la aplicación en AWS con `sam deploy --guided`

Otras utilidades de SAM CLI: `sam sync`, logs y limpieza

▼ Anatomía de plantillas de AWS y recursos generados

Anatomía de una plantilla SAM

Recurso AWS::Serverless::Function

Recurso AWS::Serverless::Function: eventos

Recurso AWS::Serverless::Function: recursos generados

Recurso AWS::Serverless::Api

Recurso AWS::Serverless::Application

Recurso AWS::Serverless::SimpleTable

Recurso AWS::Serverless::StateMachine

Ejemplos y repositorios de recursos de AWS SAM

Introsucción

AWS SAM (Serverless Application Model de AWS) es un framework de código abierto para construir aplicaciones **serverless** mediante *Infrastructure as Code*. Permite definir en archivos YAML la infraestructura serverless con sintaxis abreviada (*shorthand*) que luego se transforma en recursos reales de AWS CloudFormation durante el despliegue.

En otras palabras, **SAM extiende CloudFormation para simplificar la definición de elementos puramente serverless** como funciones Lambda, API Gateway, tablas DynamoDB, etc: una template de SAM sigue siendo una template de CloudFormation válida, simplemente incluye una línea **Transform** especial que indica a CloudFormation que aplique las transformaciones de SAM. En

runtime, CloudFormation expande las instrucciones de SAM a recursos nativos equivalentes. Esto brinda el poder de CloudFormation pero con menos código y mayor productividad.

En resumen, AWS SAM nos permite declarar la infraestructura serverless de forma declarativa y sencilla, ahorrando boilerplate y automáticamente gestionando configuraciones por nosotros.

Ventajas frente a CloudFormation puro

Al usar SAM obtenemos varias ventajas respecto a escribir plantillas puras de CloudFormation para recursos serverless.

La principal es la **brevedad**: con SAM se usan menos líneas de código YAML/JSON gracias a recursos específicos (prefijo **AWS::Serverless**). Por ejemplo, un **AWS::Serverless::Function** en **SAM crea automáticamente la función Lambda junto a su rol IAM de ejecución y cualquier trigger/evento asociado, todo en una sola declaración**. Sin SAM, tendríamos que definir manualmente un recurso **AWS::Lambda::Function**, un **AWS::IAM::Role** con políticas, y configurar por separado eventos de API Gateway o S3, etc. Con SAM eso viene implícito y simplificado.

Otra ventaja es la **cohesión**: los recursos serverless de SAM encapsulan buenas prácticas (por ejemplo despliegue gradual con `AutoPublishAlias` en funciones). Además, podemos combinar recursos SAM y de CloudFormation estándar en la misma plantilla, dándonos lo mejor de ambos mundos. En definitiva, SAM agiliza el desarrollo reduciendo la complejidad, pero sin sacrificar el acceso al “bajo nivel” cuando se requiere (podemos seguir poniendo recursos CloudFormation normales en la plantilla).

SAM como extensión de CloudFormation (Transform)

AWS SAM funciona como una extensión de CloudFormation mediante el uso de una **Transform**.

Cada plantilla SAM debe declarar `Transform: AWS::Serverless-2016-10-31` al inicio. Esta instrucción le indica a CloudFormation que primero debe aplicar la macro de transformación de SAM sobre la plantilla. Durante esa transformación, **todos los recursos con tipo `AWS::Serverless::*` se convierten en recursos de CloudFormation equivalentes** antes de la creación de la *stack*.

Por ejemplo, al desplegar una función Lambda definida con `AWS::Serverless::Function`, SAM la convertirá internamente en un `AWS::Lambda::Function` junto con el rol IAM, permisos y demás elementos necesarios. Este proceso es transparente para el desarrollador, que utiliza la sintaxis simplificada mientras que CloudFormation hace el “trabajo sucio”.

Otra diferencia reseñable es la sección **Globals**, específica de SAM (no existe en CloudFormation puro), que sirve para definir propiedades comunes por defecto para todas nuestras funciones, APIs y

tablas simples.

En resumen, **SAM es esencialmente CloudFormation + transformaciones personalizadas** para recursos serverless. Aprovecha la solidez de CloudFormation (seguimiento de estado, despliegue seguro, rollbacks, etc.) añadiendo atajos y automatizaciones específicos de aplicaciones serverless.

AWS SAM CLI

Instalación y configuración inicial

- El segundo componente principal de SAM es la **CLI (Command Line Interface)**, una herramienta que nos ayuda en todo el ciclo de vida de desarrollo.
- Existen versiones de SAM CLI para macOS, Windows y Linux
- Se puede comprobar la instalación ejecutando `sam --version` en un terminal
- AWS SAM utiliza las **mismas credenciales** que la **CLI de AWS**
 - Recordemos que se pueden configurar las credenciales de AWS mediante el comando `aws configure`
 - También es posible definir las credenciales en variables de entorno o lanzar el comando desde una instancia con un rol de ejecución definido

Inicializar un proyecto con `sam init`

Uno de los primeros comandos clave es `sam init`, que inicializa un nuevo proyecto SAM desde una plantilla predefinida. Al ejecutar `sam init` en la terminal, la CLI nos guiará de forma interactiva:

1. **Origen de la plantilla** – Podemos utilizar la opción "**AWS Quick Start Templates**" para crear un proyecto genérico que nos sirva de punto de partida
2. **Tipo de aplicación de ejemplo** – SAM ofrece múltiples plantillas de ejemplo: *Hello World* (función Lambda sencilla con API Gateway), *Procesamiento de datos*, *API serverless...*
3. **Runtime y tipo de paquete** – En este punto podemos seleccionar el runtime (Python, NodeJS, Java, etc.) y el tipo de empaquetado del código, que puede ser un fichero Zip o una imagen de contenedor
4. **Opciones adicionales** – La CLI pregunta si queremos habilitar X-Ray (herramienta de tracing), monitoreo con CloudWatch Application Insights, o logging estructurado. Algunas de esas opciones (como X-Ray o CloudWatch Application Insights) no son compatibles con los labs de AWS Academy. Es recomendable por tanto no activarlas (en todo caso, se podrían activar a posteriori)
5. **Nombre del proyecto** – Finalmente damos un nombre a la aplicación (por defecto "**sam-app**" u otro nombre descriptivo).

Después de responder estas preguntas, SAM CLI descargará la plantilla seleccionada y creará la estructura de directorios y archivos en una nueva carpeta con el nombre del proyecto.

Por ejemplo, para **"Hello World" en Python**, se creará un directorio `sam-app/` con archivos como:

- `template.yaml` (la plantilla SAM de infraestructura)
- código fuente de la función (por ej. en `hello_world/app.py`)
- archivos de configuración como `samconfig.toml` (opcional, para guardar parámetros de despliegue)
- carpetas de pruebas unitarias e integradas (`tests/`)
- un ejemplo de evento de entrada (`events/event.json`)

Al finalizar, la CLI nos muestra próximos pasos sugeridos, como por ejemplo comandos para validar la plantilla o iniciar un pipeline de integración continua (este último caso no es compatible con los labs de AWS Academy, ya que exige la creación de un rol, aunque mostraremos un ejemplo de la estructura producida en una de las prácticas). En resumen, `sam init` nos da un **proyecto listo para empezar** con una estructura organizada y código de ejemplo, acelerando la fase inicial de desarrollo.

Estructura de un proyecto SAM después de `sam init`

Una vez creado el proyecto con `sam init` , conviene revisar su estructura antes de continuar. Dentro del directorio generado veremos típicamente:

- Carpeta del código de la función (por ejemplo `hello_world/`), con el handler (`app.py` o similar) y un archivo de dependencias como `requirements.txt` o `package.json` , en función del runtime elegido.
- Archivo `template.yaml` en la raíz del proyecto, que define la infraestructura serverless (este es el corazón de SAM).
- Carpeta `tests/` con subdirectorios para pruebas unitarias y de integración, incluyendo archivos de ejemplo de eventos (útil para `sam local invoke`).
- Archivo `README.md` con instrucciones específicas de la plantilla de ejemplo.
- Archivo de configuración `samconfig.toml` (solo si decidimos guardar la configuración del despliegue interactivo, ver más adelante).

Por ejemplo, la plantilla **Hello World** en Python tendrá una función Lambda llamada `HelloWorldFunction` en la template, cuyo código está en `hello_world/app.py` . La template definirá también un evento API (mapeado a path `/hello`) que invoca a dicha función. La idea general es tener todo lo básico listo, pudiendo entrar directamente a modificar el código de la función o los recursos en la plantilla.

Construir la aplicación localmente con `sam build`

Tras haber revisado o modificado el código, el siguiente paso típico es compilar o preparar la aplicación con `sam build`, que empaqueta el código fuente Lambda con sus dependencias, dejándolo listo para el despliegue. Cuando ejecutamos `sam build` en la raíz del proyecto, la CLI realiza las siguientes **acciones**:

- Lee la `template.yaml` para ubicar las funciones y sus rutas de código (por ejemplo `CodeUri` de cada función).
- Para cada función, instala las dependencias (p.ej. `pip install -r requirements.txt` o `npm install` según runtime).
- Compila el código si es necesario (por ejemplo, para funciones en Java hay que hacer `build`).
- Coloca los resultados en una carpeta oculta llamada `.aws-sam/`. En `.aws-sam/build/` veremos una subcarpeta por cada función (con el código listo para empaquetar) y una copia de la `template` transformado (`template.yaml` actualizado apuntando a los artefactos locales).

En el [enlace a la función Hello World](#) vemos un ejemplo de salida de `sam build` que nos muestra que se crearon los artefactos y dónde. Si el *build* ha ido bien, tendremos una `template` empaquetada y lista en `.aws-sam/build/template.yaml`, que será utilizada en el despliegue para referenciar los paquetes construidos.

Pruebas locales con SAM CLI con `sam local`

Una gran ventaja de SAM CLI es la capacidad de probar funciones **localmente** antes de desplegarlas en AWS. **Para ello es necesario haber instalado Docker** en el equipo de desarrollo. Existen dos formas principales:

- `sam local invoke` - Este comando **nos permite ejecutar una función Lambda definida en nuestra template** en un entorno local similar al de AWS. Podemos pasarle un evento de prueba (por ejemplo un JSON con la entrada). Por ejemplo:
`sam local invoke HelloWorldFunction --event events/event.json`. Esto lanzará un contenedor Docker con el runtime correspondiente, cargará nuestro código y ejecutará la función con el evento proporcionado, imprimiendo el resultado y los logs en la consola. Es útil para probar funciones aisladas, como lógicas de procesamiento de datos, etc., alimentándolas con diferentes eventos de test.
- `sam local start-api` - Este comando levanta un servidor API Gateway local en `http://localhost:3000` (por defecto) que mapea las rutas definidas en nuestra `template` a las funciones Lambda correspondientes. De esta forma, si en la `template` teníamos un evento API Path: `/hello` método GET asociado a `HelloWorldFunction`, podemos simplemente hacer una petición GET a `http://127.0.0.1:3000/hello` y SAM CLI lanzará la función Lambda localmente y

nos devolverá la respuesta, [emulando todo el flujo \(request/response\)](#). Es ideal para probar APIs completas de forma local, incluyendo múltiples endpoints.

Desplegar la aplicación en AWS con `sam deploy --guided`

Este comando permite desplegar los recursos serverless en AWS. El comando `sam deploy` se encarga de empaquetar y crear/actualizar el *stack* de CloudFormation. **La primera vez es recomendable usar el asistente `--guided`, que pedirá la siguiente información:**

- **Nombre del stack** – Cómo se llamará la stack de CloudFormation (por defecto sugiere el nombre del proyecto).
- **Región de AWS** – La región donde queremos desplegar (usará la definida por defecto mediante la AWS CLI).
- **Confirmación de cambios** – Opción *Confirm changes before deploy*. Si decimos que sí, antes de aplicar los cambios nos mostrará un resumen (*changeset*) para confirmar manualmente. Durante desarrollo suele ponerse "no" para desplegar directo sin confirmación en cada iteración.
- **Permisos de creación de roles** – SAM pregunta si le permitimos crear roles IAM necesarios automáticamente. Esta opción puede dejarse en el valor por defecto aunque en los labs de Academy no podamos crear roles: si al crear funciones Lambda indicamos el rol `LabRole`, funcionará.
- **Opciones avanzadas** – Como preservar los recursos en fallo (*Disable rollback*), avisos sobre falta de autorizaciones en endpoints API, etc. Normalmente aceptamos los valores por defecto.
- **Guardar configuración** – Para indicar si queremos guardar estos parámetros en un archivo config (`samconfig.toml`) para futuras ejecuciones. Es recomendable para evitar tener que reintroducirllos cada vez.

Tras el diálogo, SAM procederá a empaquetar los artefactos.

SAM sube el código de las Lambdas a un bucket S3 (lo hace automáticamente, o podemos especificar uno), actualiza las referencias en la template (para que hagan referencia a las rutas dentro de dicho bucket), y luego llama a CloudFormation para desplegar el stack. Veremos la creación de recursos en la CLI o en el *Timeline* de consola y al final un mensaje de éxito con el ID del stack. Incluso nos dará un listado con los outputs, por ejemplo la URL del endpoint API desplegado.

En futuras ocasiones, ya podemos usar simplemente `sam deploy` (sin `--guided`), puesto que la configuración ya habrá sido guardada en el fichero `samconfig.toml`.

Al final, internamente `sam deploy` es un envoltorio que realiza un `aws cloudformation package + aws cloudformation deploy`, gestionando S3 y demás detalles, por lo que simplifica mucho el proceso.

Otras utilidades de SAM CLI: `sam sync` , logs y limpieza

- **`sam sync`** - Un comando más reciente en SAM CLI es `sam sync` . **Sirve para sincronizar cambios locales en la nube rápidamente, sin necesidad de hacer un deploy completo manual.** Por ejemplo, `sam sync --stack-name sam-app --watch` monitoriza nuestro directorio local y sube cambios incrementales automáticamente. Esto acelera el ciclo *write -> deploy -> test* en la nube durante el desarrollo, aunque suele usarse con precaución para no perder el control de los cambios desplegados.
- **`sam logs`** - Después de desplegar, una forma cómoda de ver los logs de ejecución de nuestras Lambdas es usando `sam logs -n NombreDeLaFuncion --stack-name MiStack --tail` . SAM CLI buscará en *CloudWatch Logs* los registros de la función indicada y los mostrará en la terminal en tiempo real. Esto nos evita tener que ir manualmente a la consola web de CloudWatch. Podemos utilizar filtros o en directo con `--tail` para depurar comportamientos en la nube.
- **`sam delete`** - Cuando terminemos con una aplicación y queramos **eliminar** todos los recursos de AWS que creó, podemos usar `sam delete` . Este comando nos preguntará confirmación y luego borrará la stack de CloudFormation y también el bucket de S3 donde se subieron los paquetes. Es una forma rápida de limpiar todo lo que SAM desplegó, evitando costes innecesarios. Alternativamente, siempre podemos borrar la stack manualmente desde CloudFormation, pero con `sam delete` nos aseguramos de no olvidar recursos como el bucket de artefactos.

Anatomía de plantillas de AWS y recursos generados

Anatomía de una plantilla SAM

Una plantilla SAM es esencialmente [una plantilla de CloudFormation con secciones especiales](#):

- **Transform (obligatorio):** Debe ser la primera sección e indica que se aplique la macro de SAM.
- **Globals (opcional):** Sección única de SAM donde definimos valores por defecto para recursos serverless. Por ejemplo, podemos establecer aquí que todas las funciones Lambda tengan un determinado *MemorySize* o *Timeout* por defecto, en lugar de repetirlo en cada función. Afecta a `AWS::Serverless::Function` , `AWS::Serverless::Api` y `SimpleTable` .
- **Description, Metadata:** Igual que en CloudFormation estándar, para describir la plantilla o adjuntar metadatos.
- **Parameters, Mappings, Conditions:** También igual que en CloudFormation; SAM soporta pasar parámetros, hacer *mappings* (tablas de búsqueda) y condiciones lógicas para incluir/excluir recursos según el contexto.
- **Resources (obligatorio):** Lista de recursos a crear. Aquí pueden mezclarse recursos SAM (tipo `AWS::Serverless::*`) junto con recursos normales de AWS (cualquier `AWS::Servicio::Tipo`). Por

ejemplo podemos tener `AWS::Serverless::Function` para funciones Lambda y un `AWS::S3::Bucket` estándar en la misma plantilla.

- **Outputs (opcional):** Declaración de salidas de la stack, igual que CloudFormation. Podemos hacer `Ref` o `Fn::GetAtt` tanto a recursos declarados como a recursos generados por SAM (existe cierta sintaxis para referenciar los recursos "internos" creados por la transform).

Un aspecto importante de la plantilla SAM es que **los recursos *serverless* permiten definir eventos directamente** dentro de la declaración de la función o máquina de estados, etc. Por ejemplo, en un `AWS::Serverless::Function` podemos incluir un apartado `Events:` donde listamos disparadores como HTTP (API), S3, SNS, Schedule, etc., sin tener que definir esos servicios por separado: SAM se encargará de crearlos (p. ej., crea una suscripción SNS por nosotros).

En la práctica, para empezar a escribir una plantilla SAM normalmente partimos de un snippet base que luego rellenaremos (vemos que comparte casi todo con *Cloudformation*):

```
AWSTemplateFormatVersion: '2010-09-09'
Transform: AWS::Serverless-2016-10-31
Description: >-
  Mi aplicación serverless de ejemplo
Globals:
  Function:
    Timeout: 10
    MemorySize: 128
Resources:
  # ... recursos aquí LO UNICO OBLIGATORIO ...
Outputs:
  # ... salidas aquí ...
```

Recurso `AWS::Serverless::Function`

Este es posiblemente el recurso más utilizado en SAM: representa una función Lambda junto con toda su configuración asociada. Al declarar un `AWS::Serverless::Function`, SAM va a generar automáticamente:

- La función AWS Lambda propiamente dicha (`AWS::Lambda::Function`).
- Un rol de ejecución IAM si no proporcionamos uno (con permisos básicos para escribir logs, etc.). En AWS Academy deberemos indicar siempre el rol `LabRole`.
- Las instancias de eventos/triggers que definamos bajo la función, como por ejemplo endpoints API Gateway, reglas de EventBridge (CloudWatch Events), suscripciones a colas SQS o temas SNS, etc., junto con los permisos necesarios para que esos triggers invoquen la Lambda.

- Opcionalmente, recursos para *versionado* y *alias* si especificamos propiedades como `AutoPublishAlias` (esto haría que SAM cree un `AWS::Lambda::Version` y un `AWS::Lambda::Alias` apuntando a la nueva versión, facilitando despliegues graduales)

Type: `AWS::Serverless::Function`

Properties:

`CodeUri`: ruta/al/código/

`Handler`: archivo.función_handler # (no requerido si es imagen)

`Runtime`: nodejs18.x | python3.9 | etc.

`MemorySize`: 128

`Timeout`: 5

Referencia al rol de AWS Academy

`Role`: !Sub arn:aws:iam::\${AWS::AccountId}:role/LabRole

`Environment`:

`Variables`:

`ENV_VAR`: "value"

`Events`:

`NombreEvento`:

Type: Api | Schedule | S3 | SNS | SQS | DynamoDB | etc.

`Properties`: {... según tipo ...}

Muchas propiedades coinciden con las de `AWS::Lambda::Function` (`Runtime`, `Handler`, `Memory`, `Timeout`, etc.), pero SAM introduce atajos. Por ejemplo, `Policies` nos permite adjuntar permisos comunes simplemente nombrándolos (como `AWSLambdaBasicExecutionRole` para logs en CloudWatch, o incluso `AmazonDynamoDBReadOnlyAccess` si necesita leer de tablas) sin tener que escribir el Role IAM completo. También podemos poner `Policies`: `*nombre_de_recurso*:acción*` o incluso políticas IAM inline

En el caso de **AWS Academy**, esta opción está limitada y en su lugar deberemos definir directamente la propiedad `Role` haciendo referencia al rol `LabRole` genérico.

Recurso `AWS::Serverless::Function`: eventos

Aquí es donde brilla SAM. Podemos añadir sub-secciones en *Events* para indicar qué desencadena la función. Por cada evento declarado, SAM también genera los permisos necesarios para que el servicio origen pueda invocar la función (un `AWS::Lambda::Permission`).

A continuación se muestran algunos ejemplos de eventos.

- Type: `Api` – Vincula la función con un endpoint HTTP. Podemos especificar el método (GET, POST, etc.) y el path. Si no definimos un recurso `AWS::Serverless::Api` aparte, SAM creará uno

implícitamente que agrupa todos los endpoints.

Ejemplo:

```
Events:
Hello:
  Type: Api
  Properties:
  Path: /hello
  Method: get
```

Esto hace que se cree un API Gateway REST con un recurso `/hello` y método GET que invoca esta Lambda.

- **Type: Schedule** – Programa la ejecución periódica de la función usando Amazon EventBridge (antes CloudWatch Events). Se provee una expresión de cronograma.

Ejemplo:

```
Events:
DailyTrigger:
  Type: Schedule
  Properties:
  Schedule: cron(0 8 * * ? *) # todos los días a las 08:00 UTC
  Input: '{"ping": "true"}' # opcional, payload fijo
```

Esto creará detrás de escenas una regla programada que invoca la Lambda cada día a esa hora.

- **Type: S3** – Configura que cuando ocurran eventos en un bucket S3 (ej. se sube un objeto), se dispare la función. Debemos especificar el bucket y los tipos de evento (`s3:ObjectCreated:*` típicamente).

Ejemplo:

```
Events:
FileUpload:
  Type: S3
  Properties:
  Bucket: !Ref MiBucket
  Events: s3:ObjectCreated:*
  Filter:
    S3Key:
    Rules:
      - Name: suffix
        Value: ".jpg"
```

Lo anterior indicaría que la función Lambda se ejecuta cuando se sube un archivo con sufijo .jpg al bucket referenciado.

- Type: DynamoDB – Permite suscribir la función a un *stream* de DynamoDB (para procesar cambios en tabla).
- Type: SNS / SQS – Suscribe la Lambda a un tema SNS o a una cola SQS para procesar mensajes entrantes.
- Type: EventBridge – Para eventos personalizados de un bus de eventos.

Recurso AWS::Serverless::Function: recursos generados

Conversión a CloudFormation: Como ya mencionamos, al desplegar, SAM transformará una `AWS::Serverless::Function` en **varios recursos**.

Por ejemplo, un recurso `AWS::Serverless::Function` con evento API produce:

- una `AWS::Lambda::Function`
- una `AWS::ApiGateway::RestApi` con `AWS::ApiGateway::Resource` y `AWS::ApiGateway::Method`
- un `AWS::Lambda::Permission` que permite al API invocar la lambda
- un `AWS::IAM::Role` para la Lambda (a menos que hayamos especificado uno, como para el caso de AWS Academy)

```
HelloWorldFunction:
  Type: AWS::Serverless::Function
  Properties:
    CodeUri: hello_world/
    Handler: app.lambda_handler
    Runtime: python3.9
    Events:
      Hello:
        Type: Api
        Properties:
          Path: /hello
          Method: get
```

Aquí definimos la función `HelloWorldFunction` con su código, runtime Python y la función manejadora. Tiene un evento API GET en `**/hello`. Este recurso en SAM nos crea toda la infraestructura para un "Hello World" HTTP: la URL de API Gateway, la integración con Lambda y los permisos necesarios.

En conclusión, `AWS::Serverless::Function` resume en uno solo lo que de otra forma serían múltiples recursos y configuraciones, facilitando mucho la definición de funciones Lambda y sus disparadores

en laC.

Recurso AWS::Serverless::Api

Este recurso representa una API en Amazon API Gateway (REST). En muchos casos no necesitamos declararlo explícitamente, ya que si definimos eventos API en nuestras funciones sin asociarlos a una API, SAM creará una API implícita que agrega todos esos endpoints. Sin embargo, usar un recurso `AWS::Serverless::Api` nos da más control y la posibilidad de documentar/configurar la API de manera avanzada (por ejemplo, definir un **OpenAPI** completo, CORS, autorizadores, domain custom, etc.).

Principales usos de `AWS::Serverless::Api` :

- Definir una API REST y sus rutas mediante un documento OpenAPI (por propiedades `DefinitionBody` o `DefinitionUri`).
- Aplicar configuraciones globales de API Gateway como cache, logging, manejo de CORS, autorizaciones, etc., que serían engorrosas de especificar únicamente con eventos en funciones.
- Reutilizar la misma API para múltiples funciones: podemos declarar la API y luego en las funciones referenciarla por su `Ref` (propiedad `RestApiId` en el evento Api de la función).

`MyApi:`

```
Type: AWS::Serverless::Api
Properties:
  StageName: prod
  Cors: "*" # habilitar CORS para todos
  Auth:      # podríamos definir Authorizers aquí
    DefaultAuthorizer: MyAuthorizer
    Authorizers:
      MyAuthorizer:
        ...
```

Así, las funciones que definamos podrían tener eventos definidos de esta manera:

`Events:`

```
GetItem:
  Type: Api
  Properties:
    Path: /item/{id}
    Method: get
    RestApiId: !Ref MyApi
```

Con `RestApiId` vinculamos ese endpoint a la API *MyApi* declarada. Si lo omitimos, SAM agrupará endpoints en una API implícita global (llamada típicamente algo como `<StackName>Api`).

`AWS::Serverless::Api` se transforma en un `AWS::ApiGateway::RestApi` con su `AWS::ApiGateway::Stage` correspondiente. Además, dependiendo de las propiedades, puede generar otras cosas: por ejemplo, si definimos un dominio custom con la propiedad `Domain` , SAM generará un `AWS::ApiGateway::DomainName` y sus mappings necesarios. Otro ejemplo: la propiedad `UsagePlan` en SAM API haría que se creen recursos de plan de uso y API Keys asociados.

En pocas palabras, usamos `Serverless::Api` cuando queremos un manejo más explícito de la API, especialmente para configuraciones avanzadas o documentación. Para APIs pequeñas o muy simples, definir las implícitamente vía events en las funciones suele ser suficiente. No obstante, en entornos profesionales, a menudo se opta por declarar la API para poder adjuntarle definiciones OpenAPI completas y asegurarse (por ejemplo) de añadir autorizadores a todos los endpoints. SAM incluso recomienda vigilar que cada ruta tenga autenticación, mediante hooks o políticas IAM adicionales. Así que este recurso es el **análogo serverless** de un RestApi de CloudFormation pero con atajos integrados para cosas comunes en APIs (CORS, Auth, etc.).

Recurso `AWS::Serverless::Application`

Este recurso nos permite **embebecer una aplicación serverless dentro de otra**. Básicamente sirve para incluir *nested stacks* de CloudFormation, en especial **aplicaciones publicadas en el Serverless Application Repository (SAR)** de AWS o plantillas SAM externas. En CloudFormation normal, uno puede usar `AWS::CloudFormation::Stack` para anidar plantillas; `AWS::Serverless::Application` es la versión simplificada orientada a serverless.

Usos típicos:

- Reutilizar componentes serverless existentes publicados en SAR. Por ejemplo, si AWS u otra persona publicó una aplicación (un conjunto de Lambdas + infraestructura) que resuelve cierto problema, podemos integrarla a la nuestra con este recurso.
- Modularizar nuestra propia plantilla en varias. Podríamos separar en sub-plantillas y referenciarlas vía `AWS::Serverless::Application` .

La propiedad clave es **Location**: puede ser una URL a una plantilla S3, un **ApplicationId** de SAR con su versión, o una ruta local (cuando combinamos con `sam package/deploy` que la sube). Ejemplo de uso apuntando a SAR:

```
MyAuthSystem:
  Type: AWS::Serverless::Application
  Properties:
    Location:
      ApplicationId: arn:aws:serverlessrepo:us-east-1:012345678901:applications/mi-autenticacion
      SemanticVersion: 1.0.0
    Parameters:
      Param1: Value1
      Param2: Value2
```

Esto descargaría la aplicación `mi-autenticacion v1.0.0` del Serverless Repo y la desplegaría anidada, pasándole parámetros requeridos.

Internamente, un **AWS::Serverless::Application** se convierte en un **AWS::CloudFormation::Stack** anidado. La ventaja de usar la sintaxis SAM es la facilidad de referenciar aplicaciones del SAR (ya maneja `ApplicationId`, etc.) cosa que CloudFormation puro no tiene directamente. Además SAM añade automáticamente algunas etiquetas para rastrear orígenes cuando proviene del SAR.

Este recurso es muy útil para fomentar la **reutilización y composición** de aplicaciones serverless. Por ejemplo, AWS publica muchas *SAR patterns* (en [Serverless Land](#)) que uno puede incorporar en sus soluciones con un par de líneas usando **AWS::Serverless::Application**. También en equipos grandes, se pueden encapsular componentes comunes (p.ej. un módulo de autenticación, un módulo de notificaciones) en plantillas separadas y luego integrarlos fácilmente.

En cuanto a *outputs*, SAM permite recuperar outputs de la aplicación anidada vía `Outputs.NombreOutput` en `Fn::GetAtt` o `Ref`, lo cual nos da enlace entre stacks. Por ejemplo si la app anidada exporta un ARN, podemos usar `!GetAtt MyAuthSystem.Outputs.ArnDelUsuario`.

En resumen, **AWS::Serverless::Application** nos da una forma poderosa de traer funcionalidades completas a nuestra app sin reinventar la rueda, aprovechando la naturaleza modular de CloudFormation pero con la simplicidad de SAM para aplicaciones serverless.

Recurso AWS::Serverless::SimpleTable

Este recurso provee una manera rápida de crear una tabla DynamoDB simple, con un esquema mínimo (solo clave primaria). Es útil cuando solo necesitamos almacenar datos con una clave primaria y no requerimos configuraciones avanzadas de DynamoDB. Al definir

AWS::Serverless::SimpleTable, SAM creará un recurso subyacente **AWS::DynamoDB::Table** con:

- Una clave primaria llamada **id** de tipo String por defecto (o la que indiquemos en la propiedad `PrimaryKey`).

- Modo de pago `PAY_PER_REQUEST` (a demanda) por defecto, a menos que especifiquemos `ProvisionedThroughput` para capacidad aprovisionada.
- Encriptación habilitada (SSE) si lo indicamos con `SSESpecification`, etc., pero sin opción de definir índices secundarios ni cosas complejas (para eso tendríamos que ir a `AWS::DynamoDB::Table` normal).

Básicamente está pensado para casos sencillos: por ejemplo, una tabla para almacenar sesiones, o configurar un par de tablas para una demo, sin preocuparse de todos los detalles de throughput, índices, etc. De hecho, la documentación de SAM aconseja que *para cosas avanzadas, usemos `AWS::DynamoDB::Table` normal*, incluso dentro de la plantilla SAM podemos hacerlo sin problemas. `SimpleTable` es un azúcar sintáctico.

```
UsersTable:
  Type: AWS::Serverless::SimpleTable
  Properties:
    PrimaryKey:
      Name: userId
      Type: String
    ProvisionedThroughput:
      ReadCapacityUnits: 5
      WriteCapacityUnits: 5
```

Esto crearía una tabla DynamoDB llamada (según el stack) `UsersTable` con `userId` como clave de partición, y throughput fijo de 5 RCUs/WCUs. Si omitiéramos `ProvisionedThroughput`, la capacidad sería a demanda; si omitiéramos `PrimaryKey`, sería una clave `id` de tipo `String` por defecto.

Resumiendo, **`AWS::Serverless::SimpleTable`** es una manera rápida de obtener una tabla DynamoDB funcional con mínimas opciones. Es ideal en tutoriales, ejemplos rápidos, o cuando sabemos que solo necesitamos un almacenamiento clave-valor simple. Si luego los requisitos crecen, siempre podríamos reemplazarla por un recurso DynamoDB completo para mayor control.

Recurso **`AWS::Serverless::StateMachine`**

Este recurso permite definir una **máquina de estados de Step Functions** de forma abreviada. **`AWS Step Functions` orquesta flujos de trabajo mediante un diagrama de estados**. Con `AWS::Serverless::StateMachine` podemos incluir esa definición de flujo dentro de nuestra plantilla SAM y beneficiarnos de integraciones fáciles con otros recursos. Este ejemplo define una máquina con un único estado que ejecuta una función Lambda y termina. La invocación de la máquina se realiza a través de una petición POST a un endpoint.

MyWorkflow:

Type: AWS::Serverless::StateMachine

Properties:

Definition: # Aquí iría el JSON/YAML de Amazon States Language

StartAt: Hello

States:

Hello:

Type: Task

Resource: arn:aws:lambda:...:function:HelloFunction # invocar lambda

End: true

Events:

ApiTrigger:

Type: Api

Properties:

Path: /workflow/start

Method: post

Ejemplos y repositorios de recursos de AWS SAM

- **Serverless Land Patterns**: Colección mantenida por AWS con decenas de patrones serverless listos para usar, filtrables por framework (en nuestro caso SAM) y por servicios involucrados. Cada patrón incluye la plantilla SAM (u otro IaC) y código necesario para una integración concreta entre servicios (por ejemplo " **API Gateway** -> **Lambda** -> **DynamoDB** " o " **S3** -> **Lambda** -> **SQS** ") que podemos copiar o estudiar para aprender buenas prácticas.
- **aws-samples/serverless-patterns**: repositorio de GitHub que almacena todos esos ejemplos. Allí encontramos subdirectorios para cada patrón con instrucciones de despliegue.
- **Plantillas (blueprints)** disponibles al ejecutar el comando `sam init`. Estos blueprints también son excelentes puntos de partida para explorar diferentes tipos de aplicaciones serverless.
- **Example Applications dentro de la documentación oficial de AWS SAM** Encontrarás guías paso a paso muestran cómo implementar ciertos escenarios (por ejemplo, procesar eventos de S3 con Rekognition, implementar un backend web, etc.).

En resumen, **no estamos solos**: la comunidad y AWS proporcionan muchos ejemplos testeados y probados listos para utilizar. Como único detalle, recordad modificar las plantillas para **incluir el rol de AWS Academy (LabRole)** en las **funciones serverless** y en general en todos los recursos que necesiten permisos para interactuar con otros servicios de AWS.